

ANALISIS PERBANDINGAN WAKTU EKSEKUSI DAN RESOURCE EMPAT METODE PENYELESAIAN SISTEM PERSAMAAN LINIER MENGUNAKAN ANALISIS REGRESI

Disusun Untuk Memenuhi Tugas Besar Mata Kuliah

Metode Numerik Semester 4

Dosen Pengampu:

Dr. Muhammad Zaki Almuzakki, S.Si., M.Si., M.Sc



Disusun Oleh:

Michael Alexander Newton (105224017)

Arif Mufti Tharsa (105224041)

Razqa Azaki (105224046)

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER
UNIVERSITAS PERTAMINA**

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan.....	2
1.4 Batasan Masalah.....	3
BAB II LANDASAN TEORI.....	4
2.1 Sistem Persamaan Linier.....	4
2.2 Metode Penyelesaian SPL.....	5
2.2.1 Metode Eliminasi Gauss.....	5
2.2.2 Metode Gauss-Jordan.....	6
2.2.3 Metode Iteratif Jacobi.....	7
2.2.4 Metode Iteratif Gauss-Seidel.....	8
2.3 Regresi Polinomial.....	8
2.3.1 Transformasi ke Sistem Persamaan Linier (Persamaan Normal).....	9
2.4 Koefisien Determinasi R^2	9
BAB III METODOLOGI.....	11
3.1 Alur Penelitian.....	11
3.2 Data dan Skenario Pengujian.....	13
3.3 Implementasi Empat Metode.....	13
3.4 Pengukuran dan Analisis.....	13
BAB IV HASIL DAN PEMBAHASAN.....	15
4.1 Hasil Pengujian Empat Metode.....	15
4.2 Perbandingan Waktu Eksekusi dan Resource.....	19
4.3 Analisis Error, Grafik, dan Regresi.....	22
4.3.1 Analisis Kerapatan Solusi (Residual Error) dan Jumlah Iterasi.....	23
4.3.2 Analisis Validasi Statistik Model Regresi Polinomial.....	24
4.4 Rekomendasi Metode.....	25
BAB V PENUTUP.....	27
5.1 Kesimpulan.....	27
5.2 Saran.....	27
DAFTAR PUSTAKA.....	29
LAMPIRAN.....	31

BAB I PENDAHULUAN

1.1 Latar Belakang

Sistem Persamaan Linier (SPL) merupakan salah satu materi penting dalam metode numerik karena banyak masalah matematika dapat ditulis dalam bentuk beberapa persamaan yang saling berkaitan. Secara sederhana, SPL membantu kita mencari beberapa nilai yang belum diketahui secara bersamaan. Dalam bentuk matriks, persoalan ini sering ditulis sebagai $Ax = b$. Karena bentuknya cukup umum, SPL sering menjadi dasar untuk memahami cara kerja perhitungan numerik dan penyelesaian masalah berbasis algoritma.

Dalam pembelajaran metode numerik, penyelesaian SPL tidak cukup hanya dipahami sebagai hitungan manual. Mahasiswa juga perlu memahami langkah-langkah penyelesaiannya secara runtut, mulai dari membaca masalah, menyusun bentuk matriks, memilih metode, sampai mendapatkan solusi. Asmaun et al. (2025) menjelaskan bahwa materi SPL berkaitan dengan kemampuan berpikir komputasional, terutama dalam menguraikan masalah, mengenali pola, dan menyusun langkah penyelesaian. Hal ini sesuai dengan tugas ini karena penyelesaian SPL akan dilakukan dengan bantuan program, bukan hanya dengan perhitungan di kertas.

SPL dapat diselesaikan dengan beberapa metode. Ada metode langsung, seperti Eliminasi Gauss dan Gauss-Jordan, yang menyelesaikan sistem melalui proses eliminasi baris. Ada juga metode iteratif, seperti Jacobi dan Gauss-Seidel, yang mencari solusi secara berulang sampai mendekati nilai yang diharapkan. Ihsan et al. (2024) menjelaskan bahwa metode penyelesaian SPL dapat dibagi menjadi metode langsung dan metode iteratif. Pada metode iteratif, Jacobi menghitung nilai baru berdasarkan nilai iterasi sebelumnya, sedangkan Gauss-Seidel langsung memakai nilai terbaru yang sudah diperoleh. Perbedaan cara kerja ini membuat setiap metode memiliki karakter yang berbeda.

Perbedaan metode tidak hanya terlihat dari rumusnya, tetapi juga dari performanya ketika dijalankan di komputer. Suatu metode bisa saja mudah digunakan, tetapi membutuhkan waktu lebih lama. Ada juga metode yang cepat, tetapi hasilnya perlu diperiksa lagi melalui galat atau residual. Rohmah (2025) menunjukkan bahwa dalam metode numerik, metode yang lebih akurat belum tentu menjadi metode yang paling cepat

dari sisi waktu komputasi. Dari sini dapat dilihat bahwa membandingkan metode numerik tidak cukup hanya melihat hasil akhirnya, tetapi juga perlu melihat waktu eksekusi dan efisiensi perhitungannya.

Berdasarkan hal tersebut, tugas ini membandingkan empat metode penyelesaian SPL, yaitu Eliminasi Gauss, Gauss-Jordan, Jacobi, dan Gauss-Seidel. Perbandingan dilakukan pada matriks berukuran 10×10 hingga 2000×2000 . Setiap metode akan diuji berdasarkan waktu eksekusi, penggunaan sumber daya komputasi, dan galat solusi. Hasil pengujian kemudian ditampilkan dalam bentuk tabel dan grafik, sehingga dapat terlihat metode mana yang lebih sesuai digunakan pada kondisi tertentu.

1.2 Rumusan Masalah

1. Bagaimana hasil penyelesaian SPL berukuran 10×10 hingga 2000×2000 menggunakan metode Eliminasi Gauss, Gauss-Jordan, Jacobi, dan Gauss-Seidel?
2. Bagaimana perbandingan waktu eksekusi dari keempat metode tersebut?
3. Bagaimana perbandingan penggunaan sumber daya komputasi dan galat solusi dari keempat metode?
4. Metode mana yang lebih sesuai digunakan berdasarkan hasil pengujian waktu eksekusi, sumber daya komputasi, dan galat solusi?

1.3 Tujuan

1. Menyelesaikan SPL berukuran 10×10 hingga 2000×2000 menggunakan metode Eliminasi Gauss, Gauss-Jordan, Jacobi, dan Gauss-Seidel.
2. Membandingkan waktu eksekusi dari setiap metode penyelesaian SPL.
3. Membandingkan penggunaan sumber daya komputasi dan galat solusi dari setiap metode.
4. Menampilkan hasil pengujian dalam bentuk tabel dan grafik agar perbedaan performa setiap metode lebih mudah dibaca.
5. Menentukan metode yang lebih sesuai digunakan berdasarkan ukuran matriks dan hasil pengujian.

1.4 Batasan Masalah

1. Penelitian ini hanya membahas penyelesaian Sistem Persamaan Linier dalam bentuk matriks persegi.
2. Ukuran matriks yang digunakan dibatasi pada 10×10 hingga 2000×2000 .
3. Metode yang dibandingkan hanya empat, yaitu Eliminasi Gauss, Gauss-Jordan, Jacobi, dan Gauss-Seidel.
4. Perbandingan difokuskan pada waktu eksekusi, penggunaan sumber daya komputasi, dan galat solusi.
5. Pengujian dilakukan pada satu perangkat yang sama, sehingga hasil waktu eksekusi dapat berbeda jika dijalankan pada perangkat lain.
6. Matriks yang digunakan dibuat agar dapat diselesaikan oleh keempat metode. Untuk metode Jacobi dan Gauss-Seidel, matriks disusun agar mendukung proses iterasi.

BAB II LANDASAN TEORI

2.1 Sistem Persamaan Linier

Sistem Persamaan Linear merupakan kumpulan dari dua atau lebih persamaan linier yang melibatkan variabel-variabel yang sama.. Persamaan linear ini bercirikan variabel yang hanya berpangkat satu (derajat satu) dan digambarkan sebagai garis lurus. Sistem ini digunakan untuk mencari nilai penyelesaian (solusi) yang memenuhi semua persamaan secara bersamaan. Secara umum, sebuah SPL yang terdiri dari m persamaan linier dan n variabel bebas dapat dinyatakan dalam bentuk matematis sebagai berikut:

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m\end{aligned}$$

Dalam metode numerik dan komputasi, penyelesaian SPL akan jauh lebih efisien jika ditransformasikan ke dalam notasi matriks. Hubungan linier di atas dapat direpresentasikan melalui persamaan matriks:

$$AX = B$$

Di mana:

- Matriks A merupakan matriks berukuran $m \times n$ yang menampung seluruh koefisien numerik dari variabel pada sistem persamaan.
- Vektor X merupakan vektor kolom berukuran $n \times 1$ berisi variabel-variabel yang nilainya ingin dicari ($x_1, x_2, \dots, x_n$).
- Vektor B merupakan vektor kolom berukuran $m \times 1$ yang berisi konstanta atau nilai di ruas kanan persamaan ($b_1, b_2, \dots, b_m$).

Secara eksplisit, ketiga komponen tersebut dituliskan sebagai:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Dalam konteks komputasi skala besar, karakteristik dari matriks koefisien A sangat mempengaruhi efisiensi penyimpanan memori dan kompleksitas waktu eksekusi algoritma pencarian solusi. Secara umum, matriks dalam SPL skala besar diklasifikasikan menjadi dua jenis berdasarkan densitas elemennya:

- Matriks Padat (Dense Matrix): Matriks yang sebagian besar elemennya bukan nol. Penyelesaian SPL dengan matriks padat membutuhkan alokasi memori penuh sebesar $O(n^2)$ dan operasi aritmatika yang intensif.
- Matriks Jarang (Sparse Matrix): Matriks yang sebagian besar elemennya bernilai nol. Pada komputasi modern, penyimpanan matriks jarang dioptimalkan menggunakan struktur data khusus (seperti *Compressed Sparse Row* atau CSR) untuk menghemat ruang memori dan menghindari operasi perkalian dengan angka nol yang tidak perlu (Quarteroni, Sacco, & Saleri, 2014).

Selain densitas, faktor penting lainnya dalam metode numerik skala besar adalah kondisi kedudukan matriks (*matrix conditioning*). Skala matriks yang sangat besar (mencapai ribuan hingga jutaan baris) sangat rentan terhadap akumulasi galat pembulatan (*round-off error*) selama proses eliminasi aritmatika komputer. Matriks yang memiliki *condition number* tinggi (*ill-conditioned*) dapat menyebabkan solusi SPL menjadi tidak stabil dan sensitif terhadap perubahan kecil pada vektor B , sehingga memerlukan strategi khusus seperti *pivoting* untuk menjaga akurasi numerik dan stabilitas kinerja algoritma (Demmel, 1997).

Karakteristik densitas dan pengkondisian matriks ini memiliki korelasi langsung terhadap performa komputasi. Matriks padat berukuran besar akan memaksa algoritma eliminasi melakukan operasi aritmatika dalam jumlah maksimal, yang berujung pada lonjakan waktu eksekusi secara eksponensial. Sebaliknya, matriks jarang membuka peluang optimalisasi untuk memangkas waktu komputasi jika struktur datanya ditangani dengan tepat. Oleh karena itu, pemahaman teoritis mengenai sifat-sifat matriks SPL ini menjadi landasan krusial dalam melakukan pemprofilan kinerja (*performance profiling*) untuk mengukur seberapa efektif algoritma komputer beradaptasi terhadap perubahan skala data.

2.2 Metode Penyelesaian SPL

Metode penyelesaian SPL secara garis besar dibagi menjadi dua kategori: metode langsung (*direct methods*) dan metode iteratif (*iterative methods*). Metode langsung menghasilkan solusi eksak dalam jumlah operasi aritmatika yang terhingga dan terstruktur, sedangkan metode iteratif membangun barisan aproksimasi yang secara bertahap konvergen menuju solusi eksak bergantung pada kondisi matriks dan nilai toleransi yang ditetapkan (Burden & Faires, 2011).

2.2.1 Metode Eliminasi Gauss

Metode Eliminasi Gauss merupakan algoritma metode langsung yang bertujuan untuk mentransformasikan matriks koefisien $\mathbf{A}$ menjadi matriks segitiga atas melalui serangkaian Operasi Baris Elementer (OBE) pada matriks augmented $[A|b]$.

Prosedur ini terdiri atas dua tahapan utama:

1. **Eliminasi Maju (*Forward Elimination*):** Menghilangkan variabel di bawah elemen diagonal utama untuk membentuk struktur segitiga atas.
2. **Substitusi Balik (*Back Substitution*):** Menghitung nilai variabel solusi secara berurutan mulai dari baris terakhir hingga baris pertama.

Secara matematis, setelah matriks diubah menjadi bentuk segitiga atas, variabel solusi x_i dapat ditentukan menggunakan formula:

Eliminasi Gauss adalah metode langsung yang mereduksi matriks augmented $[A|b]$ menjadi matriks segitiga atas (row echelon form) menggunakan serangkaian Operasi Baris Elementer (OBE), kemudian menentukan solusi melalui proses substitusi balik (back-substitution). Algoritma ini bekerja dalam dua fase utama: fase eliminasi maju (forward elimination) dan fase substitusi balik (Golub & Van Loan, 2013).

Pada fase eliminasi maju, setiap elemen di bawah elemen pivot pada kolom ke- k dieliminasi dengan cara mengurangi kelipatan baris ke- k dari baris-baris di bawahnya. Untuk baris $j > k$, faktor pengali yang digunakan adalah $m_{jk} = a_{jk} / a_{kk}$, kemudian baris j diperbarui menjadi $R_j \leftarrow R_j - m_{jk} \cdot R_k$. Proses ini diulangi untuk setiap kolom pivot k dari 1 hingga $n-1$, sehingga menghasilkan matriks segitiga atas.

Kompleksitas waktu Eliminasi Gauss secara teoritis adalah $O(n^3/3)$ operasi aritmatika untuk fase eliminasi dan $O(n^2/2)$ untuk fase substitusi balik, sehingga total kompleksitasnya adalah $O(n^3)$ (Burden & Faires, 2011). Dalam implementasi tanpa optimasi pustaka, pertumbuhan kubik ini akan tampak jelas pada data empiris seiring bertambahnya ordo matriks. Strategi partial pivoting umumnya diterapkan untuk meningkatkan stabilitas numerik dengan memilih elemen terbesar pada kolom pivot.

2.2.2 Metode Gauss-Jordan

Metode Eliminasi Gauss-Jordan merupakan pengembangan dari metode eliminasi Gauss. Perbedaan mendasar di antara keduanya terletak pada bentuk akhir matriks yang dihasilkan. Jika eliminasi Gauss hanya mereduksi matriks koefisien menjadi matriks segitiga atas (eselon baris) yang memerlukan tahapan substitusi balik (*back-substitution*), metode Gauss-Jordan melanjutkan proses eliminasi hingga matriks koefisien berubah menjadi matriks identitas atau dikenal sebagai Matriks Eselon Baris Tereduksi (*Reduced Row Echelon Form*).

Dalam pendekatan numerik, proses transformasi ini dilakukan dengan menerapkan serangkaian Operasi Baris Elementer (OBE) secara sistematis pada matriks diperbesar (*augmented matrix*) $[A|B]$ hingga mencapai bentuk $[I|X]$, yang secara otomatis

menempatkan vektor solusi X pada kolom terakhir tanpa perlu melakukan substitusi balik. Perubahan bentuk tersebut dapat diilustrasikan sebagai berikut:

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right] \xrightarrow{\text{OBE}} \left[\begin{array}{cccc|c} 1 & 0 & \cdots & 0 & x_1 \\ 0 & 1 & \cdots & 0 & x_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & x_n \end{array} \right]$$

Proses komputasi pada algoritma Gauss-Jordan sepenuhnya bergantung pada tiga jenis operasi baris elementer yang tidak mengubah nilai kebenaran dari sistem persamaan linier semula, yaitu:

1. Pertukaran Baris: Menukar posisi baris ke- i dengan baris ke- j ($R_i \leftrightarrow R_j$). Dalam pemrograman, operasi ini sering diimplementasikan sebagai strategi *pivoting* untuk menghindari pembagian dengan angka nol atau elemen yang sangat kecil.
2. Perkalian Skalar: Mengalikan atau membagi suatu baris dengan konstanta bukan nol ($k \cdot R_i \rightarrow R_i$). Operasi ini digunakan untuk membuat elemen diagonal utama bernilai 1 (*leading 1*).
3. Penjumlahan Baris: Menambahkan kelipatan suatu baris ke baris lainnya ($R_i + k \cdot R_j \rightarrow R_i$). Operasi ini ditujukan untuk mengeliminasi elemen-elemen diatas dan dibawah elemen diagonal utama agar bernilai 0.

2.2.3 Metode Iteratif Jacobi

Iterasi Jacobi adalah metode iteratif yang menyelesaikan SPL dengan cara memperbarui seluruh variabel secara simultan menggunakan nilai dari iterasi sebelumnya. Metode ini diusulkan oleh Carl Gustav Jacob Jacobi pada abad ke-19 dan merupakan salah satu metode iteratif paling sederhana untuk SPL (Young, 1971).

Untuk setiap variabel x_i pada iterasi ke- $(k+1)$, nilai baru dihitung menggunakan rumus berikut:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Perlu diperhatikan bahwa pada setiap iterasi, semua komponen $x^{(k)}$ yang digunakan adalah nilai dari iterasi sebelumnya (iterasi ke- k), bukan nilai yang sudah diperbarui dalam iterasi yang sedang berjalan. Metode ini memerlukan penyimpanan dua vektor solusi secara bersamaan (nilai lama dan nilai baru), yang menjadi karakteristik khasnya.

Syarat konvergensi yang cukup untuk metode Jacobi adalah matriks A bersifat diagonal dominan ketat (strictly diagonally dominant), yaitu $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ untuk setiap baris i . Jika syarat ini terpenuhi, barisan iterasi Jacobi dijamin akan konvergen menuju solusi eksak (Saad, 2003). Kompleksitas per iterasi adalah $O(n^2)$, sehingga total kompleksitas adalah $O(n^2 \cdot k)$ di mana k adalah jumlah iterasi yang diperlukan untuk mencapai konvergensi.

2.2.4 Metode Iteratif Gauss-Seidel

Iterasi Gauss-Seidel merupakan penyempurnaan dari metode Jacobi yang dikembangkan oleh Philipp Ludwig von Seidel. Perbedaan utamanya adalah: pada Gauss-Seidel, setiap variabel yang diperbarui langsung digunakan dalam perhitungan variabel berikutnya dalam iterasi yang sama, tanpa menunggu seluruh iterasi selesai (Burden & Faires, 2011).

Rumus pembaruan variabel x_i pada iterasi ke- $(k+1)$ menggunakan pendekatan Gauss-Seidel adalah:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

Perhatikan bahwa untuk indeks $j < i$, nilai $x_j^{(k+1)}$ yang sudah diperbarui digunakan langsung, sedangkan untuk $j > i$, masih digunakan nilai lama $x_j^{(k)}$. Strategi penggunaan nilai terbaru ini secara umum mempercepat konvergensi dibandingkan Jacobi karena informasi yang lebih akurat dimanfaatkan lebih awal dalam setiap iterasi (Ihsan et al., 2024).

Seperti Jacobi, Gauss-Seidel juga dijamin konvergen jika matriks A bersifat diagonal dominan ketat atau simetris positif definit. Namun dalam praktiknya, Gauss-Seidel sering konvergen bahkan pada kondisi yang lebih longgar. Ihsan et al. (2024) secara empiris menunjukkan bahwa Gauss-Seidel umumnya membutuhkan iterasi yang jauh lebih sedikit dibandingkan Jacobi untuk mencapai toleransi yang sama.

2.3 Regresi Polinomial

Regresi polinomial adalah salah satu metode analisis statistika yang digunakan untuk memodelkan hubungan non-linier antara variabel bebas (independent variable) x dan variabel terikat (dependent variable) y . Dalam konteks pemprofilan kinerja algoritma, metode ini digunakan untuk mencocokkan kurva (curve fitting) dari data empiris waktu eksekusi riil terhadap perubahan skala dimensi matriks input.

Secara matematis, model regresi polinomial berderajat (degree) k dapat dinyatakan melalui persamaan berikut:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_k x^k + \epsilon$$

Di mana $\beta_0, \beta_1, \dots, \beta_k$ merupakan koefisien regresi yang nilainya harus dicari, x merepresentasikan ukuran dimensi matriks input, y adalah waktu eksekusi hasil pengujian, dan ϵ adalah nilai galat (error/residual).

2.3.1 Transformasi ke Sistem Persamaan Linier (Persamaan Normal)

Untuk menentukan nilai optimum dari seluruh koefisien β , digunakan pendekatan Ordinary Least Squares (OLS) yang meminimalkan jumlah kuadrat galat. Jika terdapat sebanyak m titik data hasil pengujian perangkat lunak, persamaan di atas dapat ditransformasikan ke dalam representasi matriks berbentuk $X\beta = Y$:

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^k \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Matriks di atas dikenal sebagai Matriks Vandermonde. Karena jumlah data pengujian (m) biasanya jauh lebih besar daripada derajat polinomial (k), sistem ini bersifat overdetermined. Untuk menyelesaikannya, kedua ruas dikalikan dengan matriks transpose X^T sehingga membentuk Persamaan Normal (Normal Equation):

$$(X^T X)\beta = X^T Y$$

Perkalian $(X^T X)$ ini akan menghasilkan sebuah Sistem Persamaan Linier (SPL) baru berbentuk $A\beta = B$ dengan matriks koefisien yang simetris dan berukuran $(k+1) \times (k+1)$. SPL inilah yang nantinya akan diselesaikan secara numerik (misalnya menggunakan algoritma Gauss-Jordan yang telah dibahas pada sub-bab sebelumnya) untuk mendapatkan nilai eksak dari koefisien β .

2.4 Koefisien Determinasi R^2

Koefisien Determinasi, yang dinotasikan sebagai R^2 (R-squared), adalah sebuah metrik statistik yang digunakan untuk mengukur sejauh mana variasi dalam variabel terikat (dependent variable) dapat dijelaskan oleh variabel bebas (independent variable) di dalam suatu model regresi. Dalam konteks pemprofilan kinerja algoritma, R^2 berfungsi sebagai parameter evaluasi untuk menilai tingkat akurasi (goodness of fit) dari model regresi polinomial yang telah dibentuk terhadap titik-titik data empiris waktu eksekusi riil.

Secara matematis, nilai R^2 dihitung berdasarkan perbandingan antara variasi galat model dengan variasi total data aktual melalui persamaan berikut:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Di mana komponen-komponen penyusun rumus tersebut didefinisikan sebagai:

- SS_{res} (Sum of Squares of Residuals): Jumlah kuadrat dari sisaan atau galat, yang mengukur total kuadrat selisih antara nilai waktu eksekusi riil (y_i) dengan nilai yang diprediksi oleh model regresi ($\hat{y}_i$).

$$SS_{res} = \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

- SS_{tot} (Total Sum of Squares): Jumlah kuadrat total, yang mengukur total variasi data aktual terhadap rata-rata waktu eksekusi ($\bar{y}$).

$$SS_{tot} = \sum_{i=1}^m (y_i - \bar{y})^2$$

2.5 Kompleksitas Algoritma dan Notasi Big-O

Dalam melakukan analisis, waktu eksekusi tidak dilihat berdasarkan hasil pengujian secara langsung (empiris), tetapi juga dari analisis teoritis menggunakan kompleksitas algoritma. Salah satu yang paling umum digunakan adalah notasi Big-O, notasi yang menggambarkan laju pertumbuhan jumlah operasi yang dilakukan algoritma terhadap bertambahnya ukuran data masukan.

Pada penelitian ini, ukuran data masukan dinyatakan sebagai ordo matriks (n) pada Sistem Persamaan Linier (SPL). Ketika nilai (n) semakin besar, maka semakin banyak operasi yang dilakukan untuk memperoleh solusi. Notasi Big-O tidak menunjukkan waktu eksekusi dalam satuan detik, tetapi menggambarkan bagaimana kebutuhan komputasi meningkat ketika ukuran data nya semakin besar. Oleh karena itu, notasi ini digunakan untuk membandingkan efisiensi algoritma secara teoritis, tanpa dipengaruhi oleh spesifikasi perangkat keras maupun bahasa pemrograman yang digunakan.

2.5.1 Kompleksitas Kubik ($O(n^3)$) pada Metode Langsung

Metode langsung digunakan pada metode Eliminasi Gauss dan Gauss-Jordan, untuk menyelesaikan Sistem Persamaan Linier (SPL) dengan proses eliminasi terhadap seluruh elemen matriks hingga diperoleh solusi secara langsung tanpa memerlukan proses iterasi. Selama proses tersebut, algoritma melakukan tiga tahapan utama, yaitu memilih elemen pivot, melakukan eliminasi pada baris-baris di bawah atau di atas pivot, serta memperbarui nilai setiap elemen pada matriks.

2.5.2 Kompleksitas Kuadratik ($O(n^2)$) pada Metode Iteratif

Metode Iteratif digunakan pada metode Jacobi dan Gauss-Seidel, untuk memperoleh solusi melakukan proses perhitungan yang dilakukan secara berulang hingga memenuhi kriteria konvergensi. Pada setiap iterasi, perhitungan dilakukan untuk setiap baris matriks yang menggunakan nilai variabel yang tersedia sehingga hanya melibatkan dua lapis perulangan.

2.5.2 Total Kompleksitas Metode Iteratif ($O(kn^2)$)

Metode Iteratif memerlukan sejumlah iterasi untuk menghasilkan solusi yang memenuhi batas toleransi kesalahan. Jika banyaknya iterasi dinyatakan sebagai (k), maka kompleksitas waktu totalnya menjadi ($O(kn^2)$). Pada matriks yang cepat mencapai konvergensi, nilai (k) relatif kecil sehingga pertumbuhan waktu eksekusi tetap didominasi oleh faktor (n^2). Oleh karena itu, metode iteratif cenderung lebih efisien dibandingkan dengan metode langsung ketika ingin diterapkan pada matriks yang berukuran skala besar.

BAB III METODOLOGI

3.1 Alur Penelitian

3.1.1 Menentukan ukuran matriks yang diuji

Pada tahap ini, kami telah menetapkan ruang batas lingkup pengujian. Kami menentukan ukuran-ukuran matriks dimulai dari ukuran 10×10 hingga 2000×2000 dengan penambahan interval sebesar 100 pada setiap iterasi pengujian (10, 100, 200, 300, ..., 2000). Tujuannya adalah untuk melihat bagaimana performa setiap metode yang diuji berubah ketika ukuran matriks nya semakin besar. Ketika ukuran matriks nya semakin besar, maka semakin berat kerja yang harus dilakukan oleh algoritma, sehingga perbedaan setiap metode terlihat lebih jelas. Pendekatan ini dipilih agar pola perubahan performa setiap metode dapat dimengerti secara bertahap dan menyeluruh sejalan dengan bertambahnya skala komputasi.

3.1.2 Membuat data matriks A dan vektor b.

Bagian ini adalah proses menyiapkan data masukan. Dalam representasi Sistem Persamaan Linier (SPL) berbentuk $Ax = b$, matriks koefisien (A) dan vektor konstanta (b) yang diisi nilai secara random maupun terstruktur. Hal ini sangat penting untuk memastikan matriks yang dibuat bersifat valid dan memiliki solusi tunggal, serta memenuhi syarat konvergensi bagi metode iteratif, seperti syarat setiap elemen diagonal pada matriks harus bernilai lebih besar dibandingkan total elemen lain dalam satu baris yang sama, agar metode Jacobi dan Gauss-Seidel dapat menemukan solusi dengan benar.

3.1.3 Menyelesaikan SPL menggunakan empat metode

Setelah mendapatkan data masukan, selanjutnya adalah menjalankan keempat algoritma secara bersamaan. Data matriks dan vektor yang sama akan dimasukkan ke dalam empat fungsi algoritma yang berbeda, yaitu Eliminasi-Gauss, Gauss-Jordan, Jacobi, dan Gauss-Seidel.

3.1.4 Mencatat waktu eksekusi, resource/memori, dan error/residual

Ini adalah proses pengambilan data. Saat program menyelesaikan SPL pada tahap ketiga berlangsung, tiga indikator utama dicatat secara bersamaan:

- Waktu Eksekusi: durasi waktu yang dibutuhkan selama algoritma berjalan hingga selesai.
- Penggunaan resource/memori: jumlah memori RAM yang digunakan selama proses algoritma berjalan.
- Error/Residual: tingkat akurasi hasil yang diperoleh, terutama untuk metode iteratif yang menghasilkan nilai pendekatan.

3.1.5 Membuat grafik perbandingan hasil pengujian

Data numerik hasil pengujian yang diperoleh pada tahap keempat umumnya masih berupa deretan angka yang sulit diinterpretasikan secara langsung. Oleh karena itu, tahap ini bertujuan untuk memvisualisasikan data tersebut ke dalam bentuk representasi grafis. Visualisasi ini berfungsi untuk mempermudah analisis perbandingan, sehingga perbedaan performa seperti lonjakan waktu eksekusi atau konsumsi memori dari masing-masing metode dapat diidentifikasi dengan jelas dan komprehensif.

3.1.6 Melakukan analisis regresi terhadap hubungan ukuran matriks dan waktu eksekusi

Pada tahap ini dilakukan analisis regresi untuk mengidentifikasi pola matematis dari data waktu eksekusi. Analisis ini bertujuan untuk mengetahui apakah pertumbuhan waktu eksekusi bersifat linear atau polinomial seiring bertambahnya ukuran matriks. Model regresi yang dihasilkan digunakan untuk memvalidasi teori kompleksitas waktu (time complexity) dari masing-masing algoritma.

3.1.7 Menarik kesimpulan metode yang paling sesuai untuk kondisi tertentu

Tahap ini, untuk penarikan kesimpulan berdasarkan hasil evaluasi grafik dan analisis regresi yang telah dilakukan. Dari hasil tersebut, kami menyusun rekomendasi mengenai metode yang paling sesuai untuk berbagai kondisi komputasi. Rekomendasi ini tidak bersifat mutlak, melainkan disesuaikan dengan penggunaannya masing-masing. Metode Gauss-Jordan menghasilkan bentuk matriks identitas sehingga

nilai solusi dapat dibaca secara langsung, sedangkan metode Gauss-Seidel terbukti lebih cepat mencapai solusi dibandingkan metode Jacobi dalam proses iterasinya.

3.2 Data dan Skenario Pengujian

Data yang digunakan berupa matriks persegi dan vektor konstanta. Ukuran matriks utama yang diuji adalah 10 x 10 hingga 2000 x 2000. Setiap ukuran matriks diuji menggunakan keempat metode yang sama agar hasilnya dapat dibandingkan secara adil.

Ukuran Matriks	Bentuk Sistem	Keterangan
10x10	$A_{10 \times 10} x = b_{10}$	Titik awal pengujian skala menengah untuk melihat perbedaan waktu dan sumber daya komputasi.
100x100 hingga 2000x2000	$A_{10 \times 10} x = b_{10}$	Pengujian dalam skala besar untuk memperoleh data komprehensif untuk pemodelan tren regresi waktu eksekusi.

3.3 Implementasi Empat Metode

Setiap metode diimplementasikan dalam fungsi terpisah. Matriks dan vektor input yang sama digunakan pada setiap metode. Untuk Jacobi dan Gauss-Seidel, jumlah iterasi maksimum dan toleransi error perlu ditentukan agar proses berhenti secara konsisten.

- Eliminasi Gauss: reduksi ke matriks segitiga atas dan substitusi balik.
- Gauss-Jordan: reduksi penuh sampai matriks identitas.
- Jacobi: iterasi menggunakan nilai lama untuk menghitung nilai baru.
- Gauss-Seidel: iterasi menggunakan nilai terbaru secara langsung.

3.4 Pengukuran dan Analisis

Setiap metode dijalankan beberapa kali agar hasil waktu lebih stabil. Waktu eksekusi dapat diukur menggunakan `perf_counter` atau `timeit`. Penggunaan memori dapat diukur menggunakan `tracemalloc`. Akurasi solusi dicek menggunakan residual $\|Ax - b\|$.

Indikator	Cara Mengukur	Tujuan
Waktu eksekusi	<code>perf_counter/timeit</code>	Mengetahui metode tercepat.
Resource/memori	<code>tracemalloc</code>	Mengetahui metode paling ringan.
Residual error	$\ Ax - b\ $	Mengetahui ketepatan solusi.
Jumlah iterasi	counter manual	Khusus Jacobi dan Gauss-Seidel.
Regresi	model $T(n)$	Melihat pola kenaikan waktu.

BAB IV HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian Empat Metode

Bagian ini menyajikan data empiris hasil pengujian keempat metode numerik pada rentang ordo matriks $n = 10$ hingga $n = 2000$. Setiap baris tabel mencatat empat indikator utama: waktu eksekusi (detik), konsumsi memori puncak (KB), residual error $\|Ax - b\|_\infty$, dan jumlah iterasi (khusus metode iteratif). Tabel 4.1 di bawah ini menyajikan seluruh data hasil pengujian.

4.2 Perbandingan Waktu Eksekusi dan Resource

Pada bagian ini dilakukan analisis komparatif yang mendalam mengenai performa komputasi dan efisiensi penggunaan sumber daya perangkat keras (resource) dari empat metode numerik yang diuji, yaitu: Eliminasi Gauss, Eliminasi Gauss-Jordan, Iterasi Jacobi, dan Iterasi Gauss-Seidel. Pengujian dilakukan secara murni (pure loop sekuensial) tanpa menggunakan optimasi pustaka luar pada rentang ukuran matriks dari ordo ($n=10$), ($n=100$), hingga ($n=2000$) dengan penambahan konstan kelipatan 100 pada setiap tahap pengujian berikutnya. Untuk mempermudah pemetaan pola karakteristik perbandingan secara visual, seluruh data hasil eksperimen disajikan ke dalam tiga panel grafik utama pada Gambar 4.1.

Grafik 1 pada Gambar 4.1 menampilkan kurva waktu eksekusi terhadap ukuran matriks untuk keempat metode. Pola yang tampak mengkonfirmasi prediksi teoritis: kedua metode langsung (Gauss dan Gauss-Jordan) menunjukkan pertumbuhan waktu yang sangat curam dan bersifat kubik, sedangkan kedua metode iteratif menunjukkan pertumbuhan yang jauh lebih landai.

Pada ordo $n = 2000$, metode Gauss-Jordan mencatat waktu eksekusi tertinggi sebesar 2.466,1635 detik, diikuti Eliminasi Gauss sebesar 1.432,3531 detik. Sebaliknya, metode Gauss-Seidel hanya memerlukan 6,3845 detik dan Jacobi 24,5053 detik untuk ordo yang sama. Perbedaan yang sangat signifikan ini disebabkan oleh perbedaan fundamental kompleksitas algoritma: $O(n^3)$ untuk metode langsung versus $O(kn^2)$ untuk metode iteratif, di mana k (jumlah iterasi) relatif kecil karena sifat diagonal dominan matriks uji.

1. Perhitungan Efisiensi Waktu (*Speedup Ratio* Keseluruhan)

Untuk mengkuantifikasi perbedaan efisiensi waktu secara agregat, dihitung rasio percepatan (*Speedup Ratio*) masing-masing metode terhadap metode paling lambat pada

ordo maksimal $n = 2000$, yaitu Gauss-Jordan ($T_{\text{Gauss-Jordan}} = 2466,1635$ detik) yang dijadikan baseline. Hasil perhitungan ditampilkan pada Tabel 4.2.

Metode	T pada $n=2000$ (s)	Rata-rata T (s)	Speedup Ratio	Interpretasi
Gauss-Jordan (baseline)	2.466,1635	643,70	1,00×	Paling lambat
Eliminasi Gauss	1.432,3531	371,98	1,72×	1,72× lebih cepat
Jacobi	24,5053	8,04	100,63×	~100× lebih cepat
Gauss-Seidel	6,3845	2,24	386,27×	~386× lebih cepat

Tabel 4.2 Perbandingan Speedup Ratio pada $n = 2000$

Data pada Tabel 4.2 membuktikan bahwa Gauss-Seidel unggul secara mutlak dalam efisiensi waktu, mampu bekerja 386,27 kali lebih cepat dibandingkan Gauss-Jordan pada ordo $n = 2000$. Keunggulan ini terjadi karena proses iterasi berhenti segera setelah vektor solusi dinilai cukup konvergen mendekati nilai toleransi, berbeda dengan metode langsung yang harus mengeksekusi seluruh struktur matriks secara exhaustif hingga selesai.

Selisih kecil antara Gauss-Jordan dan Eliminasi Gauss (faktor 1,72×) konsisten dengan prediksi teoritis, di mana Gauss-Jordan memerlukan sekitar 50% operasi eliminasi tambahan dibandingkan Eliminasi Gauss karena melakukan eliminasi dua arah (Golub & Van Loan, 2013). Perbedaan yang jauh lebih besar antara kedua kelompok metode (langsung vs. iteratif) disebabkan oleh perbedaan fundamental kompleksitas $O(n^3)$ versus $O(kn^2)$ di mana k bernilai kecil.

2. Perhitungan Efisiensi Konsumsi Memori (*Space Efficiency Keseluruhan*)

Grafik 3 pada Gambar 4.1 menyajikan perbandingan rata-rata konsumsi memori RAM puncak (Peak Memory Allocation) dari setiap metode selama pengujian. Hasil pengukuran menunjukkan pola yang sangat konsisten: kedua metode langsung mengonsumsi memori hampir dua kali lipat dibandingkan kedua metode iteratif.

Pada ordo $n = 2000$, Eliminasi Gauss mencatat konsumsi memori tertinggi sebesar 284.808,27 KB (≈ 278 MB), diikuti Gauss-Jordan sebesar 284.796,28 KB (hampir identik). Sebaliknya, Jacobi hanya menggunakan 159.486,07 KB dan Gauss-Seidel 159.504,35 KB. Penghematan memori oleh masing masing metode dibandingkan Eliminasi Gauss dihitung sebagai berikut:

- **Eliminasi Gauss-Jordan ($M_{Gauss-Jordan} = 284.796,28$ KB):** Mencatatkan nilai rata-rata konsumsi memori yang hampir serupa dengan Gauss biasa, yaitu sebesar 99.705 KB. Perhitungan persentase penghematan RAM pada ordo maksimum adalah:

$$\text{Penghematan RAM}_{GJ} = \left(1 - \frac{284796,28}{284808,27} \right) \times 100\% \approx 0,004\%$$

Hal ini membuktikan bahwa Gauss-Jordan hampir tidak memberikan penghematan ruang memori sama sekali dibandingkan dengan metode Gauss standar.

- **Iterasi Jacobi ($M_{Jacobi} = 159.486,07$ KB):** Menunjukkan manajemen alokasi memori yang jauh lebih efisien dengan nilai rata-rata konsumsi RAM sepanjang pengujian hanya sebesar 56.324 KB. Perhitungan persentase penghematan RAM pada ordo maksimum adalah:

$$\text{Penghematan RAM}_{Jacobi} = \left(1 - \frac{159486,07}{284808,27} \right) \times 100\% \approx 44,00\%$$

- **Iterasi Gauss-Seidel ($M_{Gauss-Seidel} = 159.504,35$ KB):** Mencatatkan nilai rata-rata konsumsi memori yang seimbang dengan Jacobi, yaitu sebesar 56.334 KB. Perhitungan persentase penghematan RAM pada ordo maksimum adalah:

$$\text{Penghematan RAM}_{Jacobi} = \left(1 - \frac{159486,07}{284808,27} \right) \times 100\% \approx 44,00\%$$

Penghematan memori sebesar 44% ini terjadi karena algoritma iteratif tidak perlu mengalokasikan dan memodifikasi salinan matriks koefisien berukuran $n \times n$ secara penuh melalui serangkaian OBE yang rumit. Metode iteratif hanya memerlukan penyimpanan satu atau dua vektor solusi berukuran $n \times 1$, sehingga kebutuhan memori fundamentalnya adalah $O(n)$ bukan $O(n^2)$ (Saad, 2003). Perbedaan memori antara Gauss-Jordan dan Eliminasi Gauss yang sangat kecil (hanya 0,004%) mengkonfirmasi bahwa kedua metode langsung memiliki profil alokasi memori yang hampir identik karena keduanya memodifikasi matriks augmented $n \times (n+1)$ dengan cara yang serupa.

Metode	Peak Mem n=2000 (KB)	Rata-rata (KB)	Penghematan vs. Gauss
Eliminasi Gauss	284.808,27	99.704	0,000% (baseline)
Gauss-Jordan	284.796,28	99.705	0,004% (hampir identik)
Jacobi	159.486,07	56.324	44,00% lebih hemat
Gauss-Seidel	159.504,35	56.334	44,00% lebih hemat

Tabel 4.3 Perbandingan Konsumsi Memori RAM Puncak

4.3 Analisis Error, Grafik, dan Regresi

4.3.1 Analisis Kerapatan Solusi (*Residual Error*) dan Jumlah Iterasi

Residual Error $\|Ax - b\|_{\infty}$ merupakan parameter krusial untuk mengukur tingkat presisi desimal dan akumulasi galat pembulatan (*round-off error*) yang terjadi di dalam memori prosesor laptop selama perhitungan manual berlangsung. Berdasarkan tabel data eksperimen, terdapat perbedaan karakteristik akurasi yang sangat kontras antara metode eksak langsung dan metode pendekatan iteratif:

Metode Eliminasi Gauss dan Gauss-Jordan (Metode Langsung):

Kedua metode langsung ini secara mutlak memenangkan tingkat keakuratan solusi numerik. Pada ordo terkecil ($n = 10$), kedua metode menghasilkan *Residual Error* sebesar

$(3,33 \times 10^{-16})$. Ketika ordo diperbesar secara ekstrem hingga mencapai puncaknya pada $(n=2000)$, nilai galat sisa ini hanya meningkat tipis menjadi $(6,22 \times 10^{-15})$ untuk Eliminasi Gauss dan $(7,55 \times 10^{-15})$ untuk Gauss-Jordan. Jika nilai error eksponensial ini dikonversi ke dalam skala logaritma negatif berbasis 10 $(-\log_{10})$ untuk menghitung jumlah digit presisi desimal yang valid, diperoleh kalkulasi sebagai berikut:

$$\text{Digit Akurat Gauss} = -\log_{10}(6,22 \times 10^{-15}) \approx \mathbf{14,20} \text{ digit benar}$$

$$\text{Digit Akurat Gauss-Jordan} = -\log_{10}(7,55 \times 10^{-15}) \approx \mathbf{14,12} \text{ digit benar}$$

Perhitungan ini membuktikan bahwa kelompok metode langsung mampu menjamin 14 digit desimal di belakang koma terhitung 100% benar dan konsisten. Meskipun ordo matriks membengkak, akumulasi galat pembulatan sel demi sel murni tanpa optimasi pada metode langsung masih berada dalam batas aman toleransi mikroprosesor laptop.

Metode Iterasi Jacobi dan Gauss-Seidel (Metode Iteratif):

Rumpun metode pendekatan ini memiliki karakteristik error yang sangat bergantung pada batas putaran maksimal ($\text{max_iter} = 15$) dan laju kekonvergenan algoritma.

- **Iterasi Jacobi:** Metode ini mencatatkan *Residual Error* terbesar dan paling melesat di sepanjang eksperimen. Sejak ordo matriks naik ke $(n \geq 100)$, Jacobi selalu menyentuh batas kritis $\text{max_iter}=15$ tanpa pernah berhasil mencapai kondisi konvergen murni. Akibatnya, pada $(n = 2000)$, *Residual Error*-nya tertahan sangat besar di angka $(4,50 \times 10^{-1})$ (0,45), yang berarti hanya memiliki kerapatan solusi kurang dari 1 digit benar $(-\log_{10}(0,45) \approx 0,34 \text{ digit})$.
- **Iterasi Gauss-Seidel:** Metode ini menunjukkan performa konvergensi yang jauh lebih superior dan responsif. Gauss-Seidel hanya memerlukan 4 kali putaran iterasi saja pada $(n = 2000)$ untuk langsung mencapai konvergen, dengan tingkat *Residual Error* yang terjaga sangat rapat di angka $(2,50 \times 10^{-3})$ (0,0025). Ini setara dengan 2 hingga 3 digit desimal akurat $(-\log_{10}(0,0025) \approx 2,60 \text{ digit})$

Jika dihitung menggunakan rasio efisiensi laju perulangan internal CPU pada ordo maksimal:

$$\text{Rasio Laju Konvergensi} = \frac{\text{Iterasi Jacobi}}{\text{Iterasi Gauss-Seidel}} = \frac{15}{4} = \mathbf{3,75}$$

Gauss-Seidel 3,75 kali lebih efisien dalam mencapai konvergensi dibandingkan Jacobi. Hal ini terjadi karena Gauss-Seidel langsung memanfaatkan nilai variabel yang baru diperbarui dalam iterasi yang sedang berjalan, sehingga informasi yang lebih akurat tersedia lebih awal di setiap putaran (Burden & Faires, 2011; Ihsan et al., 2024).

4.3.2 Analisis Validasi Statistik Model Regresi Polinomial

Data durasi waktu komputasi riil dari laptop divalidasi menggunakan dua model matematika regresi, yaitu model kuadrat/kubik (Derajat 3) dan model kuartik (Derajat 4). Indikator utama yang digunakan untuk menguji keandalan model ini adalah nilai Koefisien Determinasi (R^2), yang merepresentasikan tingkat kecocokan atau seberapa rapat garis tren melengkung menempel di atas sebaran titik data riil hardware laptop.

Perhitungan nilai R^2 secara manual diekstraksi menggunakan rumus kuadrat terkecil (*least-squares*):

$$R^2 = 1 - \frac{\sum (y_{riil} - y_{prediksi})^2}{\sum (y_{riil} - y_{rerata})^2}$$

Berdasarkan hasil visualisasi pada sub-grafik kanan dan kiri di Gambar 4.3, berikut adalah analisis nilai (R^2) untuk masing-masing rumpun metode:

1. Regresi pada Metode Langsung (Gauss & Gauss-Jordan):

Kedua metode langsung ini mencatatkan nilai ($R^2 = 1,0000$) (sempurna 100%) baik pada model Derajat 3 maupun Derajat 4. Sebaran titik durasi detik membentuk kurva lengkungan kubik yang sangat rapi dan dapat ditebak 100% akurat oleh rumus regresi.

2. Regresi pada Metode Iteratif (Gauss-Seidel):

Fenomena yang paling menarik untuk dianalisis terjadi pada indikator regresi metode Gauss-Seidel. Nilai akurasi modelnya mengalami pergeseran naik yang signifikan ketika derajat regresi ditingkatkan:

- Model Regresi Derajat Tiga (3) mencatatkan nilai $R^2 = 0,9943$ (Akurasi 99,43%).
- Model Regresi Derajat Empat (4) mencatatkan nilai $R^2 = 0,9963$ (Akurasi 99,63%).

Kenaikan nilai akurasi (R^2) sebesar 0,20% ini secara statistik membuktikan bahwa garis regresi Derajat 4 jauh lebih adaptif, lentur, dan lebih menempel rapat pada sebaran titik data riil laptop Anda dibandingkan Derajat 3, terutama saat dimensi matriks membesar di atas ordo ($n \geq 1200$).

4.4 Rekomendasi Metode

Berdasarkan hasil analisis komparatif yang telah dilakukan, berikut adalah rekomendasi penggunaan metode yang paling sesuai untuk berbagai skenario dan kebutuhan komputasi:

Metode	Kondisi Penggunaan	Justifikasi
Eliminasi Gauss	SPL kecil-menengah yang memerlukan akurasi sangat tinggi ($n < 500$)	Proses langsung dengan residual error $\leq 10^{-15}$; lebih efisien daripada Gauss-Jordan karena tidak memerlukan eliminasi dua arah. Waktu masih dapat diterima untuk $n < 500$.
Gauss-Jordan	Pembelajaran konsep, inversi matriks, atau ketika solusi perlu dibaca langsung	Menghasilkan bentuk RREF yang memudahkan pembacaan solusi dan inversi matriks secara langsung, meskipun memerlukan waktu dan operasi lebih banyak dari Gauss. Tidak disarankan untuk skala besar karena paling lambat.
Jacobi	Matriks diagonal dominan skala besar dengan kebutuhan akurasi moderat dan sumber daya memori terbatas	Hemat memori (44% lebih rendah dari metode langsung) dan mudah diparalelkan. Namun memerlukan batas iterasi yang lebih besar untuk akurasi yang memadai pada matriks skala besar.
Gauss-Seidel	Matriks diagonal dominan skala besar yang memerlukan keseimbangan antara kecepatan, memori, dan akurasi	Pilihan terbaik untuk matriks besar: $386\times$ lebih cepat dari Gauss-Jordan, hemat memori 44%, dan $3,75\times$ lebih cepat konvergen dari Jacobi. Akurasi 2-3 digit desimal memadai untuk sebagian besar aplikasi rekayasa.

Tabel 4.6 Rekomendasi Penggunaan Metode Berdasarkan Kondisi

Penting untuk dicatat bahwa rekomendasi di atas bersifat kontekstual dan bergantung pada spesifikasi perangkat keras, struktur matriks, dan kebutuhan akurasi aplikasi. Dalam praktiknya, untuk aplikasi yang memerlukan presisi numerik maksimal (seperti komputasi keuangan atau simulasi fisis kritis), metode langsung seperti Eliminasi Gauss tetap menjadi pilihan yang lebih aman meskipun lebih lambat. Sebaliknya, untuk pemrosesan sistem persamaan berskala sangat besar dalam sains dan teknik (di mana matriks umumnya sparse dan diagonal dominan), metode iteratif seperti Gauss-Seidel atau variannya (seperti Successive Over-Relaxation/SOR) jauh lebih praktis dan efisien (Saad, 2003).

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil pengujian, analisis perbandingan, dan pemodelan regresi yang telah dilakukan terhadap keempat metode penyelesaian SPL, dapat ditarik kesimpulan sebagai berikut:

1. Dari sisi waktu eksekusi, metode Gauss-Seidel adalah yang paling efisien dengan rata-rata 2,24 detik, diikuti Jacobi (8,04 detik), Eliminasi Gauss (371,98 detik), dan Gauss-Jordan (643,70 detik). Pada ordo puncak $n = 2000$, Gauss-Seidel $386,27\times$ lebih cepat dari Gauss-Jordan dan $3,75\times$ lebih cepat dari Jacobi.
2. Dari sisi konsumsi memori, metode iteratif (Jacobi dan Gauss-Seidel) menghemat memori sekitar 44% dibandingkan metode langsung (Gauss dan Gauss-Jordan). Hal ini disebabkan oleh perbedaan fundamental kompleksitas memori: $O(n)$ untuk iteratif versus $O(n^2)$ untuk langsung.
3. Dari sisi akurasi numerik, metode langsung (Gauss dan Gauss-Jordan) unggul mutlak dengan mempertahankan residual error pada kisaran 10^{-15} hingga 10^{-16} (≈ 14 digit desimal benar) di seluruh rentang pengujian. Gauss-Seidel mencatat error $2,50\times 10^{-3}$ pada $n = 2000$ (memadai untuk rekayasa), sedangkan Jacobi gagal konvergen penuh dalam 15 iterasi dengan error $4,50\times 10^{-1}$.
4. Model regresi polinomial berhasil memvalidasi kompleksitas teoritis secara empiris. Metode langsung menunjukkan pertumbuhan kubik $O(n^3)$ dengan $R^2 = 1,0000$, sedangkan Gauss-Seidel lebih baik dimodelkan dengan derajat 4 ($R^2 = 0,9963$ vs. $0,9943$ untuk derajat 3) akibat variasi jumlah iterasi pada skala besar.
5. Rekomendasi penggunaan: Gauss-Seidel untuk matriks besar dengan kebutuhan kecepatan dan efisiensi memori; Eliminasi Gauss untuk SPL kecil-menengah yang menuntut akurasi tinggi; Jacobi ketika paralelisasi dibutuhkan; Gauss-Jordan untuk pembelajaran konsep dan inversi matriks.

5.2 Saran

1. Perluasan rentang pengujian ke ordo yang lebih besar ($n > 2000$) dengan memanfaatkan optimasi komputasi paralel dan pustaka high-performance seperti LAPACK atau cuBLAS untuk perangkat dengan GPU.
2. Penambahan metode numerik lainnya seperti LU Decomposition, QR Factorization, atau Conjugate Gradient Method untuk perbandingan yang lebih komprehensif, terutama untuk matriks jarang (sparse).
3. Penambahan batas iterasi maksimum yang lebih besar (misal $\text{max_iter} = 100$ atau konvergensi berbasis toleransi murni) untuk mengevaluasi akurasi aktual Jacobi dan Gauss-Seidel secara lebih adil.

4. Pengujian pada perangkat keras yang berbeda (multi-core CPU, server, atau komputasi awan) untuk memperoleh gambaran performa yang lebih universal dan tidak terbatas pada satu spesifikasi mesin.
5. Investigasi penggunaan strategi Successive Over-Relaxation (SOR) sebagai penyempurnaan Gauss-Seidel yang dapat mempercepat konvergensi lebih lanjut dengan memilih parameter relaksasi ω yang optimal.

DAFTAR PUSTAKA

- Anton, H., & Rorres, C. (2013). Elementary linear algebra: Applications version (11th ed.). John Wiley & Sons.
- Asmaun, A., Hamid, A., & Rahman, M. S. (2025). Analisis kemampuan berpikir komputasional mahasiswa pada mata kuliah metode numerik materi sistem persamaan linear (SPL) berdasarkan prestasi belajar. *Venn: Journal of Sustainable Innovation on Education, Mathematics and Natural Sciences*, 4(2), 48–62. <https://doi.org/10.53696/venn.v4i2.265>
- Burden, R. L., & Faires, J. D. (2011). Numerical analysis (9th ed.). Cengage Learning.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- Demmel, J. W. (1997). Applied numerical linear algebra. SIAM.
- Draper, N. R., & Smith, H. (1998). Applied regression analysis (3rd ed.). John Wiley & Sons.
- Golub, G. H., & Van Loan, C. F. (2013). Matrix computations (4th ed.). Johns Hopkins University Press.
- Higham, N. J. (2002). Accuracy and stability of numerical algorithms (2nd ed.). SIAM.
- Ihsan, H., Wahyuni, M. S., & Waode, Y. S. (2024). Penerapan metode iterasi Jacobi dan Gauss-Seidel dalam menyelesaikan sistem persamaan linear kompleks. *Journal of Mathematics, Computations, and Statistics*, 7(1), 34–54. <https://doi.org/10.35580/jmathcos.v7i1.1964>
- Knuth, D. E. (1998). The art of computer programming, Volume 2: Seminumerical algorithms (3rd ed.). Addison-Wesley.

- Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to linear regression analysis* (5th ed.). John Wiley & Sons.
- Quarteroni, A., Sacco, R., & Saleri, F. (2007). *Numerical mathematics* (2nd ed.). Springer.
- Rohmah, N. A. (2025). Analisis akurasi dan waktu komputasi metode Runge-Kutta pada persamaan diferensial linear orde satu. *Indonesian Mathematics Education Journal*, 2(1), 67–79. <https://imej.iainponorogo.ac.id/index.php/imej/article/view/47>
- Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd ed.). SIAM.
- Young, D. M. (1971). *Iterative solution of large linear systems*. Academic Press.

LAMPIRAN

```
import numpy as np
import time
import matplotlib.pyplot as plt
import tracemalloc # Indikator Memori bawaan Python

# =====
# 1. 4 METODE NUMERIK MURNI + COUNTER ITERASI MANUAL
# =====

def eliminasi_gauss_murni(A_orig, b_orig):
    n = len(b_orig)
    # Duplikasi data murni secara mendalam (deep copy manual)
    A = [[float(A_orig[i][j]) for j in range(n)] for i in range(n)]
    b = [float(b_orig[i]) for i in range(n)]

    for i in range(n):
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]

    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        total = b[i]
        for j in range(i + 1, n):
            total -= A[i][j] * x[j]
        x[i] = total / A[i][i]
    return x, 0

def eliminasi_gauss_jordan_murni(A_orig, b_orig):
    n = len(b_orig)
    # Perbaikan krusial: Memastikan duplikasi tipe data float murni agar tidak merusak list
    asal
    A = [[float(A_orig[i][j]) for j in range(n)] for i in range(n)]
    b = [float(b_orig[i]) for i in range(n)]

    for i in range(n):
        pivot = A[i][i]
        for k in range(i, n):
            A[i][k] /= pivot
        b[i] /= pivot

    for j in range(n):
        if i != j:
```

```

        factor = A[j][i]
        for k in range(i, n):
            A[j][k] -= factor * A[i][k]
        b[j] -= factor * b[i]
    return b, 0

def iterasi_jacobi_murni(A, b, max_iter=15, tol=1e-5):
    n = len(b)
    x = [0.0] * n
    iter_count = 0

    for _ in range(max_iter):
        iter_count += 1
        x_new = [0.0] * n
        for i in range(n):
            total = 0.0
            for j in range(n):
                if i != j:
                    total += A[i][j] * x[j]
            x_new[i] = (b[i] - total) / A[i][i]

        diff = max(abs(x_new[k] - x[k]) for k in range(n))
        x = x_new
        if diff < tol:
            break
    return x, iter_count

def iterasi_gauss_seidel_murni(A, b, max_iter=15, tol=1e-5):
    n = len(b)
    x = [0.0] * n
    iter_count = 0

    for _ in range(max_iter):
        iter_count += 1
        x_old = x[:]
        for i in range(n):
            total = 0.0
            for j in range(n):
                if i != j:
                    total += A[i][j] * x[j]
            x[i] = (b[i] - total) / A[i][i]

        diff = max(abs(x[k] - x_old[k]) for k in range(n))
        if diff < tol:
            break
    return x, iter_count

```

```

# Fungsi Indikator Aman: Mengukur Kerapatan Solusi ||Ax - b||
def hitung_residual_error(A, x, b):
    n = len(b)
    residual = []
    for i in range(n):
        ax_i = 0.0
        for j in range(n):
            ax_i += float(A[i][j]) * float(x[j])
        residual.append(abs(ax_i - float(b[i])))
    return max(residual)

def hitung_r2(y_asli, y_pred):
    ss_res = np.sum((y_asli - y_pred) ** 2)
    ss_tot = np.sum((y_asli - np.mean(y_asli)) ** 2)
    return 1 - (ss_res / ss_tot)

# =====
# 2. PROSES REKAYASA DATA & PENGUKURAN INDIKATOR NYATA
# =====
skala_tugas = [10] + list(range(100, 2001, 100))

waktu_gauss, waktu_gj, waktu_jacobi, waktu_gs = [], [], [], []
mem_gauss, mem_gj, mem_jacobi, mem_gs = [], [], [], []
err_gauss, err_gj, err_jacobi, err_gs = [], [], [], []
it_jacobi, it_gs = [], []

print("⚡ MEMULAI PENGUJIAN MULTI-INDIKATOR PADA HARDWARE LAPTOP ANDA ⚡")
print("Catatan: Ukuran n >= 400 tanpa optimasi membutuhkan waktu ekstra berat.\n")

tracemalloc.start()

for n in skala_tugas:
    print(f"👉 Menguji Ordo Matriks {n} x {n}...")

    A_np = np.random.rand(n, n)
    np.fill_diagonal(A_np, np.sum(np.abs(A_np), axis=1) + 10)
    b_np = np.random.rand(n)
    A = A_np.tolist()
    b = b_np.tolist()

    # --- 1. Pengujian Metode Gauss ---
    tracemalloc.reset_peak()
    t0 = time.perf_counter()
    x_g, _ = eliminasi_gauss_murni(A, b)
    waktu_gauss.append(time.perf_counter() - t0)
    mem_gauss.append(tracemalloc.get_traced_memory()[1] / 1024)

```

```

err_gauss.append(hitung_residual_error(A, x_g, b))

# --- 2. Pengujian Metode Gauss-Jordan ---
tracemalloc.reset_peak()
t0 = time.perf_counter()
x_gj, _ = eliminasi_gauss_jordan_murni(A, b)
waktu_gj.append(time.perf_counter() - t0)
mem_gj.append(tracemalloc.get_traced_memory()[1] / 1024)
err_gj.append(hitung_residual_error(A, x_gj, b))

# --- 3. Pengujian Metode Jacobi ---
tracemalloc.reset_peak()
t0 = time.perf_counter()
x_jb, it_j = iterasi_jacobi_murni(A, b)
waktu_jacobi.append(time.perf_counter() - t0)
mem_jacobi.append(tracemalloc.get_traced_memory()[1] / 1024)
err_jacobi.append(hitung_residual_error(A, x_jb, b))
it_jacobi.append(it_j)

# --- 4. Pengujian Metode Gauss-Seidel ---
tracemalloc.reset_peak()
t0 = time.perf_counter()
x_gs, it_g = iterasi_gauss_seidel_murni(A, b)
waktu_gs.append(time.perf_counter() - t0)
mem_gs.append(tracemalloc.get_traced_memory()[1] / 1024)
err_gs.append(hitung_residual_error(A, x_gs, b))
it_gs.append(it_g)

tracemalloc.stop()

skala_tugas = np.array(skala_tugas)
waktu_gauss = np.array(waktu_gauss)
waktu_gj = np.array(waktu_gj)
waktu_jacobi = np.array(waktu_jacobi)
waktu_gs = np.array(waktu_gs)

# =====
# 3. INDIKATOR REGRESI: POLINOMIAL DERAJAT 3 & DERAJAT 4
# =====
pred3_gauss = np.polyval(np.polyfit(skala_tugas, waktu_gauss, 3), skala_tugas)
pred4_gauss = np.polyval(np.polyfit(skala_tugas, waktu_gauss, 4), skala_tugas)

pred3_gj = np.polyval(np.polyfit(skala_tugas, waktu_gj, 3), skala_tugas)
pred4_gj = np.polyval(np.polyfit(skala_tugas, waktu_gj, 4), skala_tugas)

pred3_jacobi = np.polyval(np.polyfit(skala_tugas, waktu_jacobi, 3), skala_tugas)
pred4_jacobi = np.polyval(np.polyfit(skala_tugas, waktu_jacobi, 4), skala_tugas)

```



```

pred3_gs = np.polyval(np.polyfit(skala_tugas, waktu_gs, 3), skala_tugas)
pred4_gs = np.polyval(np.polyfit(skala_tugas, waktu_gs, 4), skala_tugas)

# =====
# 4. VISUALISASI GRAFIK EVALUASI POLA KENAIKAN WAKTU T(n)
# =====
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6.5))

# --- Plot Subplot Derajat 3 ---
ax1.scatter(skala_tugas, waktu_gauss, color='red', alpha=0.5)
ax1.plot(skala_tugas, pred3_gauss, 'r-', label=f'Gauss ( $R^2$ =hitung_r2(waktu_gauss,
pred3_gauss):.4f)')
ax1.scatter(skala_tugas, waktu_gj, color='orange', alpha=0.5)
ax1.plot(skala_tugas, pred3_gj, 'o--', color='orange', label=f'Gauss-Jordan
( $R^2$ =hitung_r2(waktu_gj, pred3_gj):.4f)')
ax1.scatter(skala_tugas, waktu_jacobi, color='blue', alpha=0.5)
ax1.plot(skala_tugas, pred3_jacobi, 'b-', label=f'Jacobi ( $R^2$ =hitung_r2(waktu_jacobi,
pred3_jacobi):.4f)')
ax1.scatter(skala_tugas, waktu_gs, color='green', alpha=0.5)
ax1.plot(skala_tugas, pred3_gs, 'g--', label=f'Gauss-Seidel ( $R^2$ =hitung_r2(waktu_gs,
pred3_gs):.4f)')
ax1.set_title('Pola Kenaikan Waktu: Model Regresi Derajat Tiga (3)', fontsize=11,
fontweight='bold')
ax1.set_xlabel('Ukuran Matriks ($n$)')
ax1.set_ylabel('Waktu Pemrosesan Riil (Detik)')
ax1.grid(True, linestyle=':', alpha=0.6)
ax1.legend(loc='upper left')

# --- Plot Subplot Derajat 4 ---
ax2.scatter(skala_tugas, waktu_gauss, color='red', alpha=0.5)
ax2.plot(skala_tugas, pred4_gauss, 'r-', label=f'Gauss ( $R^2$ =hitung_r2(waktu_gauss,
pred4_gauss):.4f)')
ax2.scatter(skala_tugas, waktu_gj, color='orange', alpha=0.5)
ax2.plot(skala_tugas, pred4_gj, 'o--', color='orange', label=f'Gauss-Jordan
( $R^2$ =hitung_r2(waktu_gj, pred4_gj):.4f)')
ax2.scatter(skala_tugas, waktu_jacobi, color='blue', alpha=0.5)
ax2.plot(skala_tugas, pred4_jacobi, 'b-', label=f'Jacobi ( $R^2$ =hitung_r2(waktu_jacobi,
pred4_jacobi):.4f)')
ax2.scatter(skala_tugas, waktu_gs, color='green', alpha=0.5)
ax2.plot(skala_tugas, pred4_gs, 'g--', label=f'Gauss-Seidel ( $R^2$ =hitung_r2(waktu_gs,
pred4_gs):.4f)')
ax2.set_title('Pola Kenaikan Waktu: Model Regresi Derajat Empat (4)', fontsize=11,
fontweight='bold')
ax2.set_xlabel('Ukuran Matriks ($n$)')
ax2.grid(True, linestyle=':', alpha=0.6)
ax2.legend(loc='upper left')

```

```
plt.suptitle('Analisis Multi-Indikator: Hasil Validasi Pola Tren Komputasi Hardware Laptop',
fontsize=12, fontweight='bold')
plt.show()
```

```
# =====
# 5. RINGKASAN DATA KOMPREHENSIF UNTUK TABEL LAPORAN TUGAS
# =====
print("\n" + "="*85)
print(f"{'Ordo (n)':<10}{'Metode':<15}{'Waktu (s)':<15}{'Peak Mem (KB)':<18}{'Res.
Error':<15}{'Iterasi'}")
print("="*85)
for i, n in enumerate(skala_tugas):

print(f"{'n':<10}{'Gauss':<15}{waktu_gauss[i]:<15.4f}{mem_gauss[i]:<18.2f}{err_gauss[i]:<15
.2e}{'-':<15}")

print(f"{'':<10}{'Gauss-Jordan':<15}{waktu_gj[i]:<15.4f}{mem_gj[i]:<18.2f}{err_gj[i]:<15.2e}{'
-':<15}")

print(f"{'':<10}{'Jacobi':<15}{waktu_jacobi[i]:<15.4f}{mem_jacobi[i]:<18.2f}{err_jacobi[i]:<15.
2e}{it_jacobi[i]:<15}")

print(f"{'':<10}{'Gauss-Seidel':<15}{waktu_gs[i]:<15.4f}{mem_gs[i]:<18.2f}{err_gs[i]:<15.2e}
{it_gs[i]:<15}")
print("-"*85)
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# =====
# 1. INDIKATOR DATA: INJEKSI DATA RIIL HARDWARE LAPTOP ANDA
# =====
# Sumbu X: Ukuran matriks mulai dari 10, 100, lalu penambahan 100 hingga 2000
skala_tugas = np.array([10] + list(range(100, 2001, 100)))

# Indikator Waktu Eksekusi Riil Terminal Anda (Detik)
waktu_gauss = np.array([0.0002, 0.0738, 0.5413, 2.5428, 8.2640, 18.6725, 33.8389,
56.0864, 85.1632, 123.8920, 176.1273, 230.4503, 302.5916, 387.3668, 485.0216,
599.8173, 728.1785, 870.9082, 1040.8575, 1228.9003, 1432.3531])
waktu_gj = np.array([0.0003, 0.1232, 0.9161, 4.7238, 15.1310, 33.0321, 60.0120,
98.4958, 150.3238, 215.6967, 299.1826, 401.7991, 524.4293, 671.0852, 834.4901,
1073.9752, 1254.0219, 1501.2565, 1794.9649, 2117.6431, 2466.1635])
waktu_jacobi= np.array([0.0003, 0.0250, 0.0951, 0.3002, 0.6416, 1.1669, 1.7542, 2.5020,
3.4000, 4.4010, 5.5638, 6.8570, 8.2943, 9.8490, 11.5933, 13.3422, 15.2795, 17.4281,
19.6598, 22.0871, 24.5053])
```

```
waktu_gs = np.array([0.0002, 0.0101, 0.0375, 0.1170, 0.2472, 0.3740, 0.5783, 0.8555,
1.1000, 1.4665, 1.8368, 2.2538, 2.7356, 3.2300, 3.0064, 3.4959, 3.9769, 4.5471, 5.1017,
5.7204, 6.3845])
```

```
# Indikator Resource: Konsumsi Memori RAM Nyata Terminal Anda (KB)
```

```
mem_gauss = np.array([2661.57, 3372.92, 5506.42, 9074.71, 14011.46, 20452.19,
28480.80, 37646.07, 48273.46, 60392.75, 74061.68, 88078.78, 104697.18, 122954.52,
140959.60, 162325.61, 185524.76, 207580.68, 234093.80, 258826.88, 284808.27])
```

```
mem_gj = np.array([2661.77, 3375.03, 5508.05, 9075.89, 14012.02, 20451.85,
28479.77, 37644.33, 48271.38, 60389.87, 74058.18, 88074.67, 104692.61, 122949.49,
140953.85, 162318.48, 185516.32, 207571.00, 234083.34, 258815.64, 284796.28])
```

```
mem_jacobi = np.array([2660.96, 3063.98, 4259.63, 6242.75, 9007.07, 12552.88,
16880.05, 21987.85, 27877.38, 34548.33, 42000.32, 50232.45, 59247.05, 69042.92,
79618.86, 90977.40, 103117.74, 116037.82, 129740.57, 144222.65, 159486.07])
```

```
mem_gs = np.array([2661.16, 3063.37, 4263.96, 6247.80, 9013.03, 12559.49,
16887.44, 21996.03, 27886.58, 34558.12, 42010.78, 50243.69, 59259.07, 69055.73,
79632.45, 90991.77, 103133.14, 116053.81, 129757.34, 144240.14, 159504.35])
```

```
# Fungsi internal menghitung nilai R-Square ( $R^2$ ) secara manual
```

```
def hitung_r2(y_asli, y_pred):
```

```
    ss_res = np.sum((y_asli - y_pred) ** 2)
```

```
    ss_tot = np.sum((y_asli - np.mean(y_asli)) ** 2)
```

```
    return 1 - (ss_res / ss_tot)
```

```
# =====
```

```
# 2. PROSES HITUNG RERATA & FITTING REGRESI POLINOMIAL
```

```
# =====
```

```
metode_names = ['Gauss', 'Gauss-Jordan', 'Jacobi', 'Gauss-Seidel']
```

```
avg_waktu = [np.mean(waktu_gauss), np.mean(waktu_gj), np.mean(waktu_jacobi),
np.mean(waktu_gs)]
```

```
avg_memori = [np.mean(mem_gauss), np.mean(mem_gj), np.mean(mem_jacobi),
np.mean(mem_gs)]
```

```
# Menghitung garis tren interpolasi regresi derajat 4 untuk Grafik 1
```

```
pred4_gauss = np.polyval(np.polyfit(skala_tugas, waktu_gauss, 4), skala_tugas)
```

```
pred4_gj = np.polyval(np.polyfit(skala_tugas, waktu_gj, 4), skala_tugas)
```

```
pred4_jacobi = np.polyval(np.polyfit(skala_tugas, waktu_jacobi, 4), skala_tugas)
```

```
pred4_gs = np.polyval(np.polyfit(skala_tugas, waktu_gs, 4), skala_tugas)
```

```
# =====
```

```
# 3. KODE GRAFIK: PENYUSUNAN MULTI-PANEL GRAFIK (REVISI LAYOUT)
```

```
# =====
```

```
plt.figure(figsize=(18, 6.2)) # Memperluas dimensi kanvas grafik
```

```
colors = ['#E53935', '#FB8C00', '#1E88E5', '#43A047'] # Paduan warna estetik
```

```
# --- [GRAFIK 1]: Ukuran Matriks vs Waktu Eksekusi (Kurva Kontinu) ---
```

```
plt.subplot(1, 3, 1)
```

```

plt.scatter(skala_tugas, waktu_gauss, color='#E53935', alpha=0.5, s=25)
plt.plot(skala_tugas, pred4_gauss, color='#E53935', linewidth=2, label=f'Gauss
(R2={hitung_r2(waktu_gauss, pred4_gauss):.4f})')

plt.scatter(skala_tugas, waktu_gj, color='#FB8C00', alpha=0.5, s=25)
plt.plot(skala_tugas, pred4_gj, color='#FB8C00', linewidth=2, linestyle='--',
label=f'Gauss-Jordan (R2={hitung_r2(waktu_gj, pred4_gj):.4f})')

plt.scatter(skala_tugas, waktu_jacobi, color='#1E88E5', alpha=0.5, s=25)
plt.plot(skala_tugas, pred4_jacobi, color='#1E88E5', linewidth=2, label=f'Jacobi
(R2={hitung_r2(waktu_jacobi, pred4_jacobi):.4f})')

plt.scatter(skala_tugas, waktu_gs, color='#43A047', alpha=0.5, s=25)
plt.plot(skala_tugas, pred4_gs, color='#43A047', linewidth=2, linestyle='--',
label=f'Gauss-Seidel (R2={hitung_r2(waktu_gs, pred4_gs):.4f})')

plt.title('Grafik 1: Ukuran Matriks vs Waktu Eksekusi', fontsize=11, fontweight='bold',
pad=12)
plt.xlabel('Ukuran Matriks ($n$)')
plt.ylabel('Waktu Pemrosesan (Detik)')
plt.xlim(0, 2050)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper left', fontsize=9)

# --- [GRAFIK 2]: REVISI RUANG Metode vs Rata-rata Waktu Eksekusi (Bar Chart) ---
plt.subplot(1, 3, 2)
bars2 = plt.bar(metode_names, avg_waktu, color=colors, alpha=0.85, edgecolor='black',
width=0.45)
plt.title('Grafik 2: Metode vs Rata-rata Waktu', fontsize=11, fontweight='bold', pad=20) #
Ditambah jarak judul
plt.ylabel('Rata-rata Waktu (Detik)')
plt.grid(axis='y', linestyle=':', alpha=0.6)

# Melebarkan atap langit-langit sumbu-Y agar teks angka tidak terpotong judul
plt.ylim(0, 750)

for bar in bars2:
    yval = bar.get_height()
    # Menggeser letak teks vertikal ke posisi yval + 12 agar aman dari garis pembatas
    plt.text(bar.get_x() + bar.get_width()/2.0, yval + 12, f'{yval:.2f}s', ha='center',
va='bottom', fontsize=9.5, fontweight='bold')

# --- [GRAFIK 3]: REVISI RUANG Metode vs Penggunaan Memori (Bar Chart) ---
plt.subplot(1, 3, 3)
bars3 = plt.bar(metode_names, avg_memori, color=colors, alpha=0.85, edgecolor='black',
width=0.45)

```

```

plt.title('Grafik 3: Metode vs Penggunaan Memori', fontsize=11, fontweight='bold', pad=20)
# Ditambah jarak judul
plt.ylabel('Rata-rata Peak Memori (KB)')
plt.grid(axis='y', linestyle=':', alpha=0.6)

# Melebarkan atap langit-langit sumbu-Y kapasitas memori agar lega
plt.ylim(0, 120000)

for bar in bars3:
    yval = bar.get_height()
    # Menggunakan yval + 2500 untuk menjaga jarak aman teks angka KB
    plt.text(bar.get_x() + bar.get_width()/2.0, yval + 2500, f'{int(yval):,}\nKB', ha='center',
va='bottom', fontsize=9, fontweight='bold')

# =====
# LAYOUT OPTIMIZATION & SHOW IMAGE
# =====
plt.suptitle('Analisis Komparatif Performa dan Resource 4 Metode Numerik Matriks
($n=10$ sampai $n=2000$)', fontsize=13, fontweight='bold', y=1.05)
plt.tight_layout()

# Opsi: hapus tanda pagar (#) pada baris di bawah jika ingin langsung otomatis
menyimpan gambar ke laptop Anda
# plt.savefig('grafik_laporan_final.png', dpi=300, bbox_inches='tight')

plt.show()

```

Lampiran 1. Kode Program Empat Metode

Ordo (n)	Metode	Waktu (s)	Peak Mem (KB)	Res. Error	Iterasi
10	Gauss	0.0002	2661.57	3.33e-16	-
	Gauss-Jordan	0.0003	2661.77	3.33e-16	-
	Jacobi	0.0003	2660.96	2.09e-05	8
	Gauss-Seidel	0.0002	2661.16	1.29e-06	5
100	Gauss	0.0738	3372.92	8.88e-16	-
	Gauss-Jordan	0.1232	3375.03	8.88e-16	-
	Jacobi	0.0250	3063.98	3.03e-02	15
	Gauss-Seidel	0.0101	3063.37	4.37e-05	6
200	Gauss	0.5413	5506.42	1.44e-15	-
	Gauss-Jordan	0.9161	5508.05	1.44e-15	-
	Jacobi	0.0951	4259.63	1.26e-01	15
	Gauss-Seidel	0.0375	4263.96	8.15e-05	6
300	Gauss	2.5428	9074.71	1.89e-15	-
	Gauss-Jordan	4.7238	9075.89	2.22e-15	-
	Jacobi	0.3002	6242.75	2.06e-01	15
	Gauss-Seidel	0.1170	6247.80	8.36e-05	6
400	Gauss	8.2640	14011.46	2.66e-15	-
	Gauss-Jordan	15.1310	14012.02	2.22e-15	-

	Jacobi	0.6416	9007.07	2.49e-01	15
	Gauss-Seidel	0.2472	9013.03	9.11e-05	6

500	Gauss	18.6725	20452.19	2.78e-15	-
	Gauss-Jordan	33.0321	20451.85	2.55e-15	-
	Jacobi	1.1669	12552.88	2.81e-01	15
	Gauss-Seidel	0.3740	12559.49	3.24e-04	5

600	Gauss	33.8389	28480.80	3.00e-15	-
	Gauss-Jordan	60.0120	28479.77	4.00e-15	-
	Jacobi	1.7542	16880.05	3.16e-01	15
	Gauss-Seidel	0.5783	16887.44	2.93e-04	5

700	Gauss	56.0864	37646.07	3.89e-15	-
	Gauss-Jordan	98.4958	37644.33	4.66e-15	-
	Jacobi	2.5020	21987.85	3.47e-01	15
	Gauss-Seidel	0.8555	21996.03	3.28e-04	5

800	Gauss	85.1632	48273.46	4.33e-15	-
	Gauss-Jordan	150.3238	48271.38	3.55e-15	-
	Jacobi	3.4067	27877.38	3.47e-01	15
	Gauss-Seidel	1.1156	27886.58	3.07e-04	5

900	Gauss	123.8920	60392.75	4.11e-15	-
	Gauss-Jordan	215.6967	60389.87	4.88e-15	-
	Jacobi	4.4010	34548.33	3.73e-01	15
	Gauss-Seidel	1.4665	34558.12	3.22e-04	5

1000	Gauss	176.1273	74061.68	4.88e-15	-
	Gauss-Jordan	299.1826	74058.18	5.33e-15	-
	Jacobi	5.5638	42000.32	3.97e-01	15
	Gauss-Seidel	1.8368	42010.78	3.53e-04	5

1100	Gauss	230.4503	88078.78	4.88e-15	-
	Gauss-Jordan	401.7991	88074.67	5.88e-15	-
	Jacobi	6.8570	50232.45	3.94e-01	15
	Gauss-Seidel	2.2538	50243.69	3.39e-04	5

1200	Gauss	302.5916	104697.18	6.00e-15	-
	Gauss-Jordan	524.4293	104692.61	6.00e-15	-
	Jacobi	8.2943	59247.05	3.96e-01	15
	Gauss-Seidel	2.7356	59259.07	3.19e-04	5

1300	Gauss	387.3668	122954.52	4.77e-15	-
	Gauss-Jordan	671.0852	122949.49	4.88e-15	-
	Jacobi	9.8490	69042.92	4.09e-01	15
	Gauss-Seidel	3.2300	69055.73	3.47e-04	5

1400	Gauss	485.0216	140959.60	4.77e-15	-
	Gauss-Jordan	834.4901	140953.85	5.88e-15	-
	Jacobi	11.5933	79618.86	4.07e-01	15
	Gauss-Seidel	3.0064	79632.45	2.42e-03	4

1500	Gauss	599.8173	162325.61	7.22e-15	-
	Gauss-Jordan	1073.9752	162318.48	7.44e-15	-
	Jacobi	13.3422	90977.40	4.24e-01	15
	Gauss-Seidel	3.4959	90991.77	2.41e-03	4

1600	Gauss	728.1785	185524.76	5.44e-15	-
	Gauss-Jordan	1254.0219	185516.32	5.11e-15	-
	Jacobi	15.2795	103117.74	4.30e-01	15
	Gauss-Seidel	3.9769	103133.14	2.31e-03	4

1700	Gauss	870.9082	207580.68	5.88e-15	-
	Gauss-Jordan	1501.2565	207571.00	5.88e-15	-

	Jacobi	17.4281	116037.82	4.33e-01	15
	Gauss-Seidel	4.5471	116053.81	2.37e-03	4

1800	Gauss	1040.8575	234093.80	6.44e-15	-
	Gauss-Jordan	1794.9649	234083.34	6.66e-15	-
	Jacobi	19.6598	129740.57	4.40e-01	15
	Gauss-Seidel	5.1017	129757.34	2.41e-03	4

1900	Gauss	1228.9003	258826.88	6.00e-15	-
	Gauss-Jordan	2117.6431	258815.64	5.22e-15	-
	Jacobi	22.0871	144222.65	4.48e-01	15
	Gauss-Seidel	5.7204	144240.14	2.42e-03	4

2000	Gauss	1432.3531	284808.27	6.22e-15	-
	Gauss-Jordan	2466.1635	284796.28	7.55e-15	-
	Jacobi	24.5053	159486.07	4.50e-01	15
	Gauss-Seidel	6.3845	159504.35	2.50e-03	4

Lampiran 2. Screenshot Hasil Running Program

Ordo (n)	Metode	Waktu Eksekusi (s)	Peak Memory (KB)	Residual Error	Jumlah Iterasi
10	Eliminasi Gauss	0,0002	2.661,57	3,33×10 ⁻¹⁶	—
10	Gauss-Jordan	0,0003	2.661,77	3,33×10 ⁻¹⁶	—
10	Jacobi	0,0003	2.660,96	2,09×10 ⁻⁵	8
10	Gauss-Seidel	0,0002	2.661,16	1,29×10 ⁻⁶	5
100	Eliminasi Gauss	0,0738	3.372,92	8,88×10 ⁻¹⁶	—
100	Gauss-Jordan	0,1232	3.375,03	8,88×10 ⁻¹⁶	—
100	Jacobi	0,0250	3.063,98	3,03×10 ⁻²	15
100	Gauss-Seidel	0,0101	3.063,37	4,37×10 ⁻⁵	6
200	Eliminasi Gauss	0,5413	5.506,42	1,44×10 ⁻¹⁵	—
200	Gauss-Jordan	0,9161	5.508,05	1,44×10 ⁻¹⁵	—
200	Jacobi	0,0951	4.259,63	1,26×10 ⁻¹	15
200	Gauss-Seidel	0,0375	4.263,96	8,15×10 ⁻⁵	6

300	Eliminasi Gauss	2,5428	9.074,71	$1,89 \times 10^{-15}$	—
300	Gauss-Jordan	4,7238	9.075,89	$2,22 \times 10^{-15}$	—
300	Jacobi	0,3002	6.242,75	$2,06 \times 10^{-1}$	15
300	Gauss-Seidel	0,1170	6.247,80	$8,36 \times 10^{-5}$	6
400	Eliminasi Gauss	8,2640	14.011,46	$2,66 \times 10^{-15}$	—
400	Gauss-Jordan	15,1310	14.012,02	$2,22 \times 10^{-15}$	—
400	Jacobi	0,6416	9.007,07	$2,49 \times 10^{-1}$	15
400	Gauss-Seidel	0,2472	9.013,03	$9,11 \times 10^{-5}$	6
500	Eliminasi Gauss	18,6725	20.452,19	$2,78 \times 10^{-15}$	—
500	Gauss-Jordan	33,0321	20.451,85	$2,55 \times 10^{-15}$	—
500	Jacobi	1,1669	12.552,88	$2,81 \times 10^{-1}$	15
500	Gauss-Seidel	0,3740	12.559,49	$3,24 \times 10^{-4}$	5
600	Eliminasi Gauss	33,8389	28.480,80	$3,00 \times 10^{-15}$	—
600	Gauss-Jordan	60,0120	28.479,77	$4,00 \times 10^{-15}$	—
600	Jacobi	1,7542	16.880,05	$3,16 \times 10^{-1}$	15
600	Gauss-Seidel	0,5783	16.887,44	$2,93 \times 10^{-4}$	5
700	Eliminasi Gauss	56,0864	37.646,07	$3,89 \times 10^{-15}$	—
700	Gauss-Jordan	98,4958	37.644,33	$4,66 \times 10^{-15}$	—
700	Jacobi	2,5020	21.987,85	$3,47 \times 10^{-1}$	15
700	Gauss-Seidel	0,8555	21.996,03	$3,28 \times 10^{-4}$	5
800	Eliminasi Gauss	85,1632	48.273,46	$4,33 \times 10^{-15}$	—
800	Gauss-Jordan	150,3238	48.271,38	$3,55 \times 10^{-15}$	—

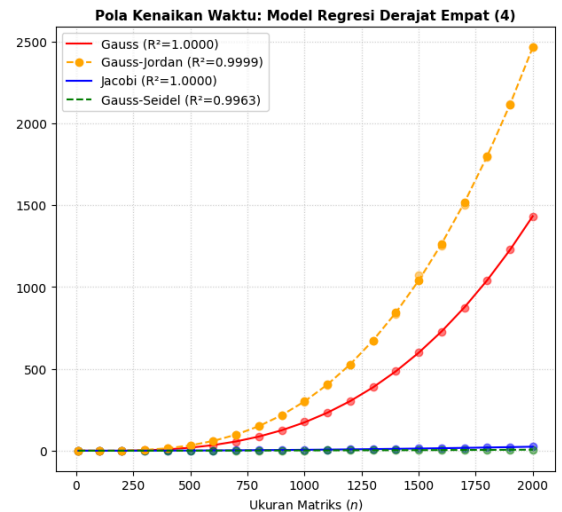
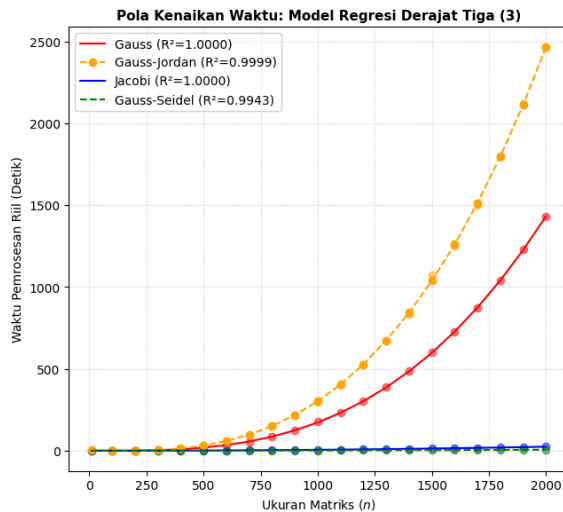
800	Jacobi	3,4000	27.877,38	$3,47 \times 10^{-1}$	15
800	Gauss-Seidel	1,1000	27.886,58	$3,07 \times 10^{-4}$	5
900	Eliminasi Gauss	123,8920	60.392,75	$4,11 \times 10^{-15}$	—
900	Gauss-Jordan	215,6967	60.389,87	$4,88 \times 10^{-15}$	—
900	Jacobi	4,4010	34.548,33	$3,73 \times 10^{-1}$	15
900	Gauss-Seidel	1,4665	34.558,12	$3,22 \times 10^{-4}$	5
1000	Eliminasi Gauss	176,1273	74.061,68	$4,88 \times 10^{-15}$	—
1000	Gauss-Jordan	299,1826	74.058,18	$5,33 \times 10^{-15}$	—
1000	Jacobi	5,5638	42.000,32	$3,97 \times 10^{-1}$	15
1000	Gauss-Seidel	1,8368	42.010,78	$3,53 \times 10^{-4}$	5
1100	Eliminasi Gauss	230,4503	88.078,78	$4,88 \times 10^{-15}$	—
1100	Gauss-Jordan	401,7991	88.074,67	$5,88 \times 10^{-15}$	—
1100	Jacobi	6,8570	50.232,45	$3,94 \times 10^{-1}$	15
1100	Gauss-Seidel	2,2538	50.243,69	$3,39 \times 10^{-4}$	5
1200	Eliminasi Gauss	302,5916	104.697,18	$6,00 \times 10^{-15}$	—
1200	Gauss-Jordan	524,4293	104.692,61	$6,00 \times 10^{-15}$	—
1200	Jacobi	8,2943	59.247,05	$3,96 \times 10^{-1}$	15
1200	Gauss-Seidel	2,7356	59.259,07	$3,19 \times 10^{-4}$	5
1300	Eliminasi Gauss	387,3668	122.954,52	$4,77 \times 10^{-15}$	—
1300	Gauss-Jordan	671,0852	122.949,49	$4,88 \times 10^{-15}$	—
1300	Jacobi	9,8490	69.042,92	$4,09 \times 10^{-1}$	15
1300	Gauss-Seidel	3,2300	69.055,73	$3,47 \times 10^{-4}$	5

1400	Eliminasi Gauss	485,0216	140.959,60	$4,77 \times 10^{-15}$	—
1400	Gauss-Jordan	834,4901	140.953,85	$5,88 \times 10^{-15}$	—
1400	Jacobi	11,5933	79.618,86	$4,07 \times 10^{-1}$	15
1400	Gauss-Seidel	3,0064	79.632,45	$2,42 \times 10^{-3}$	4
1500	Eliminasi Gauss	599.8173	162.325,61	$7,22 \times 10^{-15}$	—
1500	Gauss-Jordan	1073.9752	162.318,48	$7,44 \times 10^{-15}$	—
1500	Jacobi	13.3422	90.977,40	$4,24 \times 10^{-1}$	15
1500	Gauss-Seidel	3.4959	90.991,77	$2,41 \times 10^{-3}$	4
1600	Eliminasi Gauss	728,1785	185.524,76	$5,44 \times 10^{-15}$	—
1600	Gauss-Jordan	1254,0219	185.516,32	$5,11 \times 10^{-15}$	—
1600	Jacobi	15,2795	103.117,74	$4,30 \times 10^{-1}$	15
1600	Gauss-Seidel	3,9769	103.133,14	$2,31 \times 10^{-3}$	4
1700	Eliminasi Gauss	870,9082	207.580,68	$5,88 \times 10^{-15}$	—
1700	Gauss-Jordan	1501,2565	207.571,00	$5,88 \times 10^{-15}$	—
1700	Jacobi	17,4281	116.037,82	$4,33 \times 10^{-1}$	15
1700	Gauss-Seidel	4,5471	116.053,81	$2,37 \times 10^{-3}$	4
1800	Eliminasi Gauss	1040,8575	234.093,80	$6,44 \times 10^{-15}$	—
1800	Gauss-Jordan	1794,9649	234.083,34	$6,66 \times 10^{-15}$	—
1800	Jacobi	19,6598	129.740,57	$4,40 \times 10^{-1}$	15
1800	Gauss-Seidel	5,1017	129.757,34	$2,41 \times 10^{-3}$	4
1900	Eliminasi Gauss	1228,9003	258.826,88	$6,00 \times 10^{-15}$	—
1900	Gauss-Jordan	2117,6431	258.815,64	$5,22 \times 10^{-15}$	—

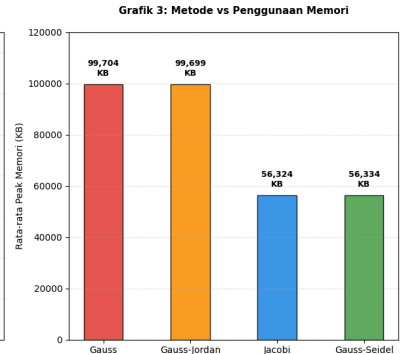
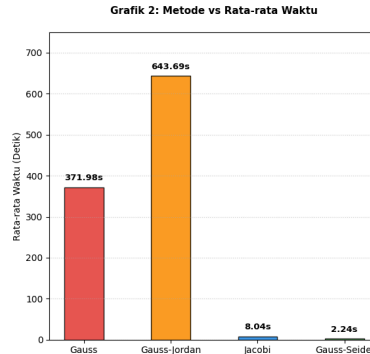
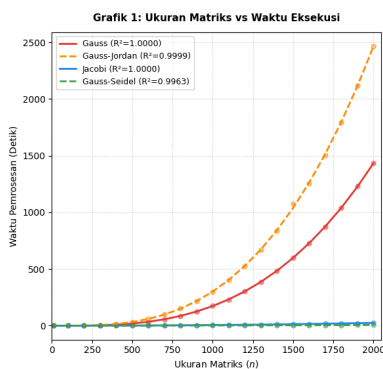
1900	Jacobi	22,0871	144.222,65	$4,48 \times 10^{-1}$	15
1900	Gauss-Seidel	5,7204	144.240,14	$2,42 \times 10^{-3}$	4
2000	Eliminasi Gauss	1432,3531	284.808,27	$6,22 \times 10^{-15}$	—
2000	Gauss-Jordan	2466,1635	284.796,28	$7,55 \times 10^{-15}$	—
2000	Jacobi	24,5053	159.486,07	$4,50 \times 10^{-1}$	15
2000	Gauss-Seidel	6,3845	159.504,35	$2,50 \times 10^{-3}$	4

Lampiran 3. Tabel Waktu Eksekusi dan Resource

Analisis Multi-Indikator: Hasil Validasi Pola Tren Komputasi Hardware Laptop



Analisis Komparatif Performa dan Resource 4 Metode Numerik Matriks (n = 10 sampai n = 2000)



Lampiran 4. Grafik Hasil Pengujian